BUP Lost & Found System

2. Overall Description

2.1 Product Perspective

The BUP Lost & Found System is a new, self-contained web application. It is not a modification of an existing system. It follows a standard three-tier architecture:

- **Presentation Layer:** A single-page application built with React and Vite, styled with a distinctive ocean-blue "sonar" theme.
- **Application Layer:** A REST API built with Node.js and Express, handling authentication, business logic, and the item-matching algorithm.
- **Data Layer:** A MySQL relational database storing users, categories, locations, lost items, found items, returned records, and notifications.

2.2 Product Functions

At a high level, the system provides the following functions:

- Student self-registration and name-based login with duplicate-name handling.
- Password-only administrator login.
- Reporting a lost item with guided dropdown-and-suggestion input fields.
- Reporting a found item anonymously, generating a private token number.
- Browsing lost and found posts, filterable by category, with or without logging in.
- An administrator workspace that pairs found items against lost items within the same category and ranks candidates by an automated match score.
- Confirming a match, which archives both posts as "returned" and notifies the item's owner.
- A searchable, live-filtering directory of all registered students.
- Optional, self-service password changes for both students and administrators.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
Student	Registers, logs in, reports lost/found items, browses posts, receives notifications	Basic computer/smartphone literacy

User Class	Description	Technical Expertise
Administrator	Manages category matching workspace, confirms returns, oversees the platform	Comfortable with web-based admin tools
Guest / Visitor	Anyone browsing "All Posts" or the student directory without an account	None required

2.4 Operating Environment

- Client: Any modern desktop or mobile web browser (Chrome, Firefox, Edge, Safari) with JavaScript enabled.
- Server: Node.js (v18+) runtime hosting the Express REST API.
- Database: MySQL 8.x, accessed via a connection pool.
- Deployment: Designed to run on a single institutional server or standard cloud hosting; no specialized hardware required.

2.5 Design and Implementation Constraints

- The system must run entirely on the specified stack: React/Vite, Node.js/Express, and MySQL.
- Passwords must be stored only as bcrypt hashes; plaintext passwords are never persisted.
- Session authentication uses stateless JWTs; the server does not maintain server-side session storage.
- Category, location, size, and color reference data is seeded centrally so dropdown and suggestion inputs remain consistent across the application.

2.6 Assumptions and Dependencies

- Users have access to a device with an internet connection and a modern web browser.
- The university provides (or the admin manually maintains) an accurate list of departments and campus locations.
- The system assumes good-faith use; identity verification of "founders" claiming a token is currently a manual, in-person administrative process.
- Future integration with the university’s official student database, email/SMS gateways, and image uploads are out of scope for the current version but are anticipated as enhancements.